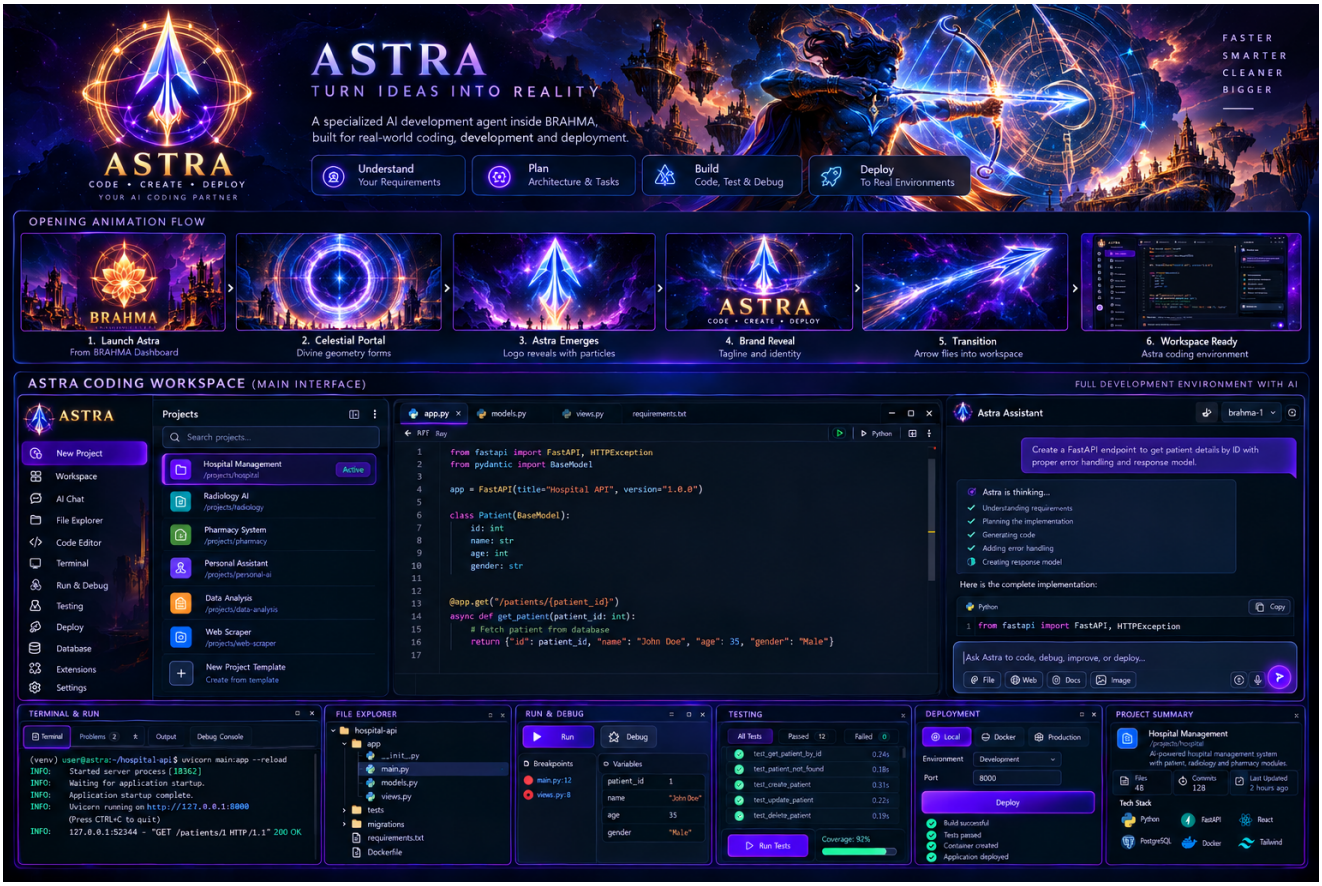
ASTRA

COMPLETE DESIGN SPECIFICATION v1

Celestial identity • Opening animation • Coding workspace • Security



Status: ASTRA design candidate for review. BRAHMA remains locked; ASTRA's logo, animation timing, and workspace become fixed only after your approval.

ASTRA — Complete Design Specification v1

Product role: ASTRA is BRAHMA's specialized coding and development workspace.

Design status: Review candidate. BRAHMA is approved and locked; ASTRA's exact visual assets and animation timing require your approval before implementation.

1. Identity and relationship to BRAHMA

• **Name:** ASTRA

• **Descriptor:** `CODE · CREATE · DEPLOY`

• **Role:** A dedicated coding agent and development environment inside the BRAHMA product family.

• **Visual inspiration:** Indian mythology's celestial-weapon/astral-arrow symbolism, expressed as abstract geometry and light. Do not depict a named deity as an exact religious figure unless separately approved.

• **Relationship:** ASTRA opens as a distinct workspace, not a renamed BRAHMA chat screen.

• **No unapproved changes:** Once ASTRA's final design is approved, logo, colors, layout, and animation are frozen unless the user explicitly requests a change.

2. Proposed ASTRA visual system

These are proposed ASTRA-specific tokens for review. They do not change BRAHMA's locked tokens.

Token	Value	Use
`astra.bg`	`#070A18`	Main IDE background
`astra.surface`	`#0D1428`	Sidebar and panels
`astra.panel`	`#101B35`	Elevated work panels
`astra.violet`	`#8B5CF6`	Primary interactive accent
`astra.electricBlue`	`#38BDF8`	Code/technical highlights
`astra.cyan`	`#22D3EE`	Status and secondary actions
`astra.gold`	`#F6C87A`	Restrained mythic highlight and shared family accent
`astra.text`	`#F7F7FB`	Primary text
`astra.muted`	`#A7AEC2`	Secondary text
`astra.border`	`#263657`	Panel borders
`astra.success`	`#22C55E`	Passing checks
`astra.warning`	`#F59E0B`	Warnings
`astra.error`	`#EF4444`	Failures
`font.ui`	`Poppins`	UI, matching BRAHMA family typography
`font.code`	`JetBrains Mono`	Editor, terminal, diffs, logs

ASTRA's primary visual character is deep indigo, electric blue, and violet. Gold is used sparingly to connect it to BRAHMA, not as the main IDE interaction color. Use centralized CSS variables; validate contrast in the actual UI.

3. Logo direction

• Create a dedicated ASTRA vector mark: a vertical celestial arrow/astral spear silhouette inside a precise circular yantra-like orbit.

• Use a sharp central arrowhead, restrained geometric rings, and a controlled violet-to-electric-blue glow.

• Wordmark: ASTRA, with `CODE · CREATE · DEPLOY` beneath it.

• Do not reuse BRAHMA's lotus as the ASTRA logo.

• Production asset deliverables: SVG primary, monochrome SVG, small-size app icon, favicon, and motion-ready vector layers.

• The generated concept poster is not the final logo source file; redraw/construct a clean vector from the approved reference before implementation.

4. Opening animation storyboard

Concept duration: approximately 5–7 seconds, skippable after the brand reveal. Timing below is a proposal for review, not yet frozen.

Handoff (0.0–0.6 s): User selects ASTRA from BRAHMA. BRAHMA content dims smoothly; no abrupt white flash.

Celestial field (0.6–1.6 s): A dark indigo field appears. Fine orbit lines and a circular yantra-like geometry trace in violet and blue.

Energy gathers (1.6–2.6 s): Light points converge along a vertical axis; a symbolic celestial arrow silhouette forms. Use abstract energy rather than an explicit deity depiction.

Emblem reveal (2.6–3.6 s): Arrow and orbit align into the ASTRA mark. The ASTRA wordmark and `CODE · CREATE · DEPLOY` appear.

Arrow transition (3.6–4.6 s): A narrow light trail moves forward and the emblem resolves into the workspace frame; no intense strobing or screen shake.

Workspace materialization (4.6–6.0 s): Project explorer, editor, assistant, and terminal panels fade/slide into their final positions.

Ready state (only when ready): Show workspace-ready status only after project context and required services have initialized. If initialization fails, show a useful error and retry option.

Motion and accessibility rules

• Provide a visible Skip action after the emblem reveal.

• Respect `prefers-reduced-motion`: replace the cinematic sequence with a short crossfade and static emblem.

• Sound is off by default; if ever added, provide a visible mute control and do not make sound necessary to understand the sequence.

• Do not use flashing/strobing effects.

• Do not hold users in an animation while backend initialization is already complete; do not claim readiness while it is still loading.

• Use the same animation duration and easing across repeated launches; avoid randomizing the brand reveal.

5. Workspace information architecture

Main shell

• **Left rail:** ASTRA mark; New Project; Workspace; AI Chat; File Explorer; Code Editor; Terminal; Run & Debug; Testing; Deploy; Database; Extensions; Settings.

• **Project pane:** project list/search, repository trust state, branch/status if Git is connected, file tree.

• **Editor center:** tabs, syntax-highlighted editor, line numbers, diagnostics, search, and diff view.

• **AI assistant pane:** task prompt, clarifying questions, proposed plan, progress, generated patch, tool activity, cancel control.

• **Bottom panel:** Terminal, Problems, Output, Debug Console, Tests.

• **Context/header:** current project/repository, runtime/environment status, branch when available, and safe run controls.

• **Responsive behavior:** wide desktop is the primary target; tablet may collapse panels; mobile is a constrained review/monitoring mode rather than a full desktop IDE.

6. Core screens

Welcome/project chooser: create project, open recent project, import repository, trust warning, supported templates.

Project import/create: path or repository source, branch selection where applicable, dependency/environment detection, trust confirmation.

Main coding workspace: explorer, editor, AI assistant, terminal, run/debug, tests.

Task planning: goal summary, questions, planned files, ordered actions, estimated risk, approval step.

Patch/diff review: file-by-file diff, accept/reject, revert, and a summary of effects.

Run & debug: start/stop, breakpoints, variables, logs, error explanation.

Testing: selected tests, run status, pass/fail, duration, output, rerun.

Git/repository: status, diff, branch, commit preparation; never silently commit or push.

Environment/dependencies: detected runtime, dependency install proposal, lockfile changes, environment errors.

Deployment: target, environment, changed resources, preview, explicit confirmation and result.

Permissions and audit: requested tool permissions, sandbox status, recent commands, cancellations, logs.

Settings and shortcuts: editor preferences, theme, key bindings, AI preferences, environment options.

Shared states: empty, loading, no repository trust, command blocked, timeout, tests failed, offline, and service error.

7. ASTRA agent workflow

User states a coding goal.

ASTRA inspects only the selected, permitted project context.

ASTRA asks clarifying questions if requirements are ambiguous.

ASTRA presents an implementation plan and identifies risky actions.

User approves the plan or edits it.

ASTRA proposes code changes as a patch/diff.

User reviews and accepts or rejects changes.

Tests run in an isolated, disposable execution environment.

ASTRA reports actual results, logs, and any unresolved failures.

Deployment or destructive actions require separate explicit confirmation.

The UI must distinguish proposed changes from applied changes, and passed tests from tests that were not run.

8. Security and trust boundaries

• Run generated code only in an isolated disposable runner with CPU, memory, PID, disk, and time limits.

• Enforce filesystem and network boundaries; no unrestricted host OS access.

• Require confirmation for destructive file changes, dependency installation with material risk, commits/pushes, secret access, and deployment.

• Show the exact command and target before approval.

• Keep credentials server-side; redact secrets from logs and model context.

• Treat repository files, comments, README instructions, and tool output as untrusted content; do not let them override system policy or tool permissions.

• Log commands, approvals, cancellations, and results; provide cancellation and timeout handling.

• Production deployment must never occur silently.

9. Proposed component states

• Buttons: default, hover, focus, disabled, loading.

• Assistant task: queued, planning, awaiting approval, editing, testing, succeeded, failed, cancelled.

• Test row: pending, running, passed, failed, skipped.

• Diff: pending review, accepted, rejected, reverted.

• Environment: ready, initializing, blocked, failed.

• Deployment: draft, awaiting confirmation, deploying, succeeded, failed, rolled back where supported.

10. Acceptance gate

Before ASTRA UI development:

• Approve the exact ASTRA vector mark and app icon.

• Approve the proposed ASTRA-specific palette and type rules.

• Approve the storyboard frames, duration, skip behavior, reduced-motion behavior, and sound policy.

• Approve the workspace layout and all core screens.

• Approve the security/confirmation behavior for tool execution.

• Use screenshot comparisons at agreed desktop/tablet widths; verify keyboard navigation and contrast.

• Record any deviation as a proposal and wait for explicit approval before changing the design.

11. Implementation alignment

• Frontend: React + JavaScript; no TypeScript files.

• Application API: Node.js + Express.

• AI orchestration/coding services: Python + FastAPI.

• Database: PostgreSQL; Redis for queue/cache/rate limiting.

• Code execution: isolated disposable runner with strict resource and network controls.

• ASTRA is a separate experience within the BRAHMA product family, with distinct routing and its own workspace state.

• This document is a design and behavior specification; it does not claim that the product or animation has been implemented.